Experiment 09 — From Scratch: LN 7×7 vs BN 7×7 vs LN 3×3 vs BN 3×3

Config	Norm	DWConv	Pretrain
09a	LayerNorm	7×7	❌
09b	BatchNorm	7×7	❌
09c	LayerNorm	3×3	❌
09d	BatchNorm	3×3	❌

Extension of Finding 1: apakah gap LN vs BN mengecil tanpa pretrained weights?

0. Setup

In [1]:
```
from google.colab import drive
drive.mount('/content/drive')

import os, shutil

# ── Sesuaikan path dataset di Drive ──────────────────────────────────────
DATASET_DRIVE = '/content/drive/MyDrive/rice-convnext/dataset'
DATASET_LOCAL = '/content/dataset'
DRIVE_DIR     = '/content/drive/MyDrive/rice-convnext/09_Scratch_LN7x7_BN3x3
os.makedirs(DRIVE_DIR, exist_ok=True)

if not os.path.exists(DATASET_LOCAL):
    print('Copying dataset ke local SSD...')
    shutil.copytree(DATASET_DRIVE, DATASET_LOCAL)
    print('Done.')
else:
    print('Local dataset sudah ada.')
```

```
Mounted at /content/drive
Copying dataset ke local SSD...
Done.
```

In [2]:
```
import random, numpy as np, pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import classification_report, confusion_matrix

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader, random_split, Subset
from torchvision import datasets, transforms, models
```

```
SEED = 42
random.seed(SEED); np.random.seed(SEED)
torch.manual_seed(SEED); torch.cuda.manual_seed(SEED)
torch.backends.cudnn.deterministic = True

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Device: {DEVICE}')
```

```
Device: cuda
```

1. Configuration

In [3]:

```
IMG_SIZE      = 224
BATCH_SIZE    = 64
NUM_EPOCHS    = 30
LR_INIT       = 1e-3
LR_FINAL      = 1e-6
WEIGHT_DECAY  = 0.05
NUM_WORKERS   = 2
IMAGENET_MEAN = [0.485, 0.456, 0.406]
IMAGENET_STD  = [0.229, 0.224, 0.225]

CONFIGS = {
    '09a': {'name': 'LN 7×7 | No Pretrain', 'norm': 'LN', 'kernel': 7,
            'save': f'{DRIVE_DIR}/09a_ln7x7.pt'},
    '09b': {'name': 'BN 7×7 | No Pretrain', 'norm': 'BN', 'kernel': 7,
            'save': f'{DRIVE_DIR}/09b_bn7x7.pt'},
    '09c': {'name': 'LN 3×3 | No Pretrain', 'norm': 'LN', 'kernel': 3,
            'save': f'{DRIVE_DIR}/09c_ln3x3.pt'},
    '09d': {'name': 'BN 3×3 | No Pretrain', 'norm': 'BN', 'kernel': 3,
            'save': f'{DRIVE_DIR}/09d_bn3x3.pt'},
}
print('Configs:', list(CONFIGS.keys()))
```

```
Configs: ['09a', '09b', '09c', '09d']
```

2. Data Loading

In [4]:

```
train_tf = transforms.Compose([
    transforms.RandomResizedCrop(IMG_SIZE),
    transforms.RandomHorizontalFlip(),
    transforms.RandomVerticalFlip(),
    transforms.RandomRotation(30),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.2),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])
val_tf = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(IMG_SIZE),
    transforms.ToTensor(),
    transforms.Normalize(IMAGENET_MEAN, IMAGENET_STD),
])
```

```
base        = datasets.ImageFolder(DATASET_LOCAL)
CLASS_NAMES = base.classes
NUM_CLASSES = len(CLASS_NAMES)
N           = len(base)
n_train, n_val = int(0.6 * N), int(0.2 * N)
n_test      = N - n_train - n_val

gen = torch.Generator().manual_seed(SEED)
tr_idx, val_idx, te_idx = [
    s.indices for s in random_split(base, [n_train, n_val, n_test], generatc
]

train_ds = Subset(datasets.ImageFolder(DATASET_LOCAL, transform=train_tf), t
val_ds   = Subset(datasets.ImageFolder(DATASET_LOCAL, transform=val_tf),   v
test_ds  = Subset(datasets.ImageFolder(DATASET_LOCAL, transform=val_tf),   t

train_loader = DataLoader(train_ds, BATCH_SIZE, shuffle=True,  num_workers=N
val_loader   = DataLoader(val_ds,   BATCH_SIZE, shuffle=False, num_workers=N
test_loader  = DataLoader(test_ds,  BATCH_SIZE, shuffle=False, num_workers=N

print(f'Classes: {NUM_CLASSES} | Train: {len(train_ds)} | Val: {len(val_ds)}
```

```
Classes: 7 | Train: 2011 | Val: 670 | Test: 672
```

3. Architecture Helpers

In [5]:

```
class BatchNormNHWC(nn.Module):
    """BatchNorm2d that accepts NHWC input (ConvNeXt block format)."""
    def __init__(self, num_features):
        super().__init__()
        self.bn = nn.BatchNorm2d(num_features)
    def forward(self, x):
        return self.bn(x.permute(0, 3, 1, 2)).permute(0, 2, 3, 1)


def replace_layernorm_with_batchnorm(module):
    for name, child in module.named_children():
        if type(child).__name__ == 'LayerNorm2d':
            setattr(module, name, nn.BatchNorm2d(child.normalized_shape[0]))
        elif isinstance(child, nn.LayerNorm):
            setattr(module, name, BatchNormNHWC(child.normalized_shape[0]))
        else:
            replace_layernorm_with_batchnorm(child)


def replace_dwconv_kernel(module, new_kernel=3):
    for name, child in module.named_children():
        if (isinstance(child, nn.Conv2d)
                and child.kernel_size == (7, 7)
                and child.groups == child.in_channels):
            setattr(module, name, nn.Conv2d(
                child.in_channels, child.out_channels,
                kernel_size=new_kernel, padding=new_kernel // 2,
                groups=child.groups, bias=child.bias is not None,
            ))
```

```
        else:
            replace_dwconv_kernel(child, new_kernel)


def build_model(norm='LN', kernel=7):
    model = models.convnext_tiny(weights=None)
    model.classifier[2] = nn.Linear(model.classifier[2].in_features, NUM_CLA
    if kernel != 7:
        replace_dwconv_kernel(model, new_kernel=kernel)
    if norm == 'BN':
        replace_layernorm_with_batchnorm(model)
    return model.to(DEVICE)


print('Architecture helpers defined.')
```

```
Architecture helpers defined.
```

4. Training & Evaluation

In [6]:

```
def get_scheduler(opt):
    warmup = optim.lr_scheduler.LinearLR(opt, start_factor=0.01, end_factor=
    cosine = optim.lr_scheduler.CosineAnnealingLR(opt, T_max=NUM_EPOCHS - 1,
    return optim.lr_scheduler.SequentialLR(opt, schedulers=[warmup, cosine],


def run_epoch(model, loader, criterion, optimizer=None):
    training = optimizer is not None
    model.train() if training else model.eval()
    total_loss, correct, total = 0.0, 0, 0
    ctx = torch.enable_grad() if training else torch.no_grad()
    with ctx:
        for images, labels in loader:
            images, labels = images.to(DEVICE), labels.to(DEVICE)
            if training:
                optimizer.zero_grad()
            out  = model(images)
            loss = criterion(out, labels)
            if training:
                loss.backward(); optimizer.step()
            total_loss += loss.item() * images.size(0)
            correct    += out.max(1)[1].eq(labels).sum().item()
            total      += labels.size(0)
    return total_loss / total, correct / total


def train(cfg_key, cfg):
    model     = build_model(norm=cfg['norm'], kernel=cfg['kernel'])
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.AdamW(model.parameters(), lr=LR_INIT, weight_decay=WEI
    scheduler = get_scheduler(optimizer)
    best_acc  = 0.0
    history   = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc

    print(f'\n[{cfg_key}] {cfg["name"]}')
```

```
    print('-' * 68)
    for epoch in range(1, NUM_EPOCHS + 1):
        tl, ta = run_epoch(model, train_loader, criterion, optimizer)
        vl, va = run_epoch(model, val_loader,   criterion)
        scheduler.step()
        history['train_loss'].append(tl); history['train_acc'].append(ta)
        history['val_loss'].append(vl);   history['val_acc'].append(va)
        if va > best_acc:
            best_acc = va
            torch.save(model.state_dict(), cfg['save'])
        print(f'  Epoch {epoch:02d}/{NUM_EPOCHS} | '
              f'Train Loss: {tl:.4f} Acc: {ta:.4f} | '
              f'Val Loss: {vl:.4f} Acc: {va:.4f} | Best: {best_acc:.4f}')

    return model, history


def evaluate(model, cfg):
    model.load_state_dict(torch.load(cfg['save'], map_location=DEVICE))
    model.eval()
    preds, labels = [], []
    with torch.no_grad():
        for imgs, lbls in test_loader:
            preds.extend(model(imgs.to(DEVICE)).max(1)[1].cpu().numpy())
            labels.extend(lbls.numpy())
    report = classification_report(labels, preds, target_names=CLASS_NAMES,
    cm     = confusion_matrix(labels, preds)
    print(f'  Accuracy: {report["accuracy"]:.4f} | F1: {report["weighted avg
    return {
        'accuracy': report['accuracy'],
        'f1'      : report['weighted avg']['f1-score'],
        'report'  : report,
        'cm_norm' : cm.astype('float') / cm.sum(axis=1)[:, np.newaxis],
        'preds'   : preds, 'labels': labels,
    }


print('Training functions defined.')
```

```
Training functions defined.
```

5. Run All 4 Experiments

In [7]:
```
# ── 09a: LN 7×7 ──────────────────────────────────────────────────────
model_a, history_a = train('09a', CONFIGS['09a'])
print('\nEvaluating 09a...')
results_a = evaluate(model_a, CONFIGS['09a'])
```

```
[09a] LN 7×7 | No Pretrain
──────────────────────────────────────────────────────────────────────────────
  Epoch 01/30 | Train Loss: 1.5721 Acc: 0.3521 | Val Loss: 1.4000 Acc: 0.420
9 | Best: 0.4209
  Epoch 02/30 | Train Loss: 1.9673 Acc: 0.3153 | Val Loss: 1.5380 Acc: 0.322
4 | Best: 0.4209
  Epoch 03/30 | Train Loss: 1.4918 Acc: 0.3471 | Val Loss: 1.3956 Acc: 0.413
4 | Best: 0.4209
  Epoch 04/30 | Train Loss: 1.4218 Acc: 0.3943 | Val Loss: 1.2520 Acc: 0.446
3 | Best: 0.4463
  Epoch 05/30 | Train Loss: 1.4105 Acc: 0.3710 | Val Loss: 1.2809 Acc: 0.447
8 | Best: 0.4478
  Epoch 06/30 | Train Loss: 1.3323 Acc: 0.4182 | Val Loss: 1.3256 Acc: 0.404
5 | Best: 0.4478
  Epoch 07/30 | Train Loss: 1.2913 Acc: 0.4296 | Val Loss: 1.2802 Acc: 0.397
0 | Best: 0.4478
  Epoch 08/30 | Train Loss: 1.2679 Acc: 0.4396 | Val Loss: 1.2838 Acc: 0.409
0 | Best: 0.4478
  Epoch 09/30 | Train Loss: 1.2576 Acc: 0.4490 | Val Loss: 1.1333 Acc: 0.489
6 | Best: 0.4896
  Epoch 10/30 | Train Loss: 1.2665 Acc: 0.4510 | Val Loss: 1.2327 Acc: 0.446
3 | Best: 0.4896
  Epoch 11/30 | Train Loss: 1.2221 Acc: 0.4694 | Val Loss: 1.1156 Acc: 0.477
6 | Best: 0.4896
  Epoch 12/30 | Train Loss: 1.2165 Acc: 0.4973 | Val Loss: 1.1297 Acc: 0.497
0 | Best: 0.4970
  Epoch 13/30 | Train Loss: 1.1619 Acc: 0.5072 | Val Loss: 1.0933 Acc: 0.507
5 | Best: 0.5075
  Epoch 14/30 | Train Loss: 1.1419 Acc: 0.4963 | Val Loss: 1.0381 Acc: 0.538
8 | Best: 0.5388
  Epoch 15/30 | Train Loss: 1.1467 Acc: 0.5231 | Val Loss: 1.0928 Acc: 0.525
4 | Best: 0.5388
  Epoch 16/30 | Train Loss: 1.1362 Acc: 0.5281 | Val Loss: 1.2289 Acc: 0.443
3 | Best: 0.5388
  Epoch 17/30 | Train Loss: 1.1472 Acc: 0.5152 | Val Loss: 1.1512 Acc: 0.455
2 | Best: 0.5388
  Epoch 18/30 | Train Loss: 1.1061 Acc: 0.5291 | Val Loss: 1.0143 Acc: 0.582
1 | Best: 0.5821
  Epoch 19/30 | Train Loss: 1.0655 Acc: 0.5460 | Val Loss: 1.0116 Acc: 0.546
3 | Best: 0.5821
  Epoch 20/30 | Train Loss: 1.0505 Acc: 0.5460 | Val Loss: 1.0002 Acc: 0.537
3 | Best: 0.5821
  Epoch 21/30 | Train Loss: 1.0305 Acc: 0.5619 | Val Loss: 0.9930 Acc: 0.585
1 | Best: 0.5851
  Epoch 22/30 | Train Loss: 1.0469 Acc: 0.5500 | Val Loss: 1.0172 Acc: 0.534
3 | Best: 0.5851
  Epoch 23/30 | Train Loss: 1.0441 Acc: 0.5450 | Val Loss: 1.0279 Acc: 0.555
2 | Best: 0.5851
  Epoch 24/30 | Train Loss: 1.0082 Acc: 0.5599 | Val Loss: 0.9480 Acc: 0.604
5 | Best: 0.6045
  Epoch 25/30 | Train Loss: 1.0155 Acc: 0.5778 | Val Loss: 0.9513 Acc: 0.579
1 | Best: 0.6045
  Epoch 26/30 | Train Loss: 1.0069 Acc: 0.5679 | Val Loss: 0.9709 Acc: 0.588
1 | Best: 0.6045
  Epoch 27/30 | Train Loss: 0.9855 Acc: 0.5843 | Val Loss: 0.9470 Acc: 0.583
6 | Best: 0.6045
```

```
  Epoch 28/30 | Train Loss: 0.9972 Acc: 0.5828 | Val Loss: 0.9481 Acc: 0.585
1 | Best: 0.6045
  Epoch 29/30 | Train Loss: 0.9964 Acc: 0.5758 | Val Loss: 0.9446 Acc: 0.592
5 | Best: 0.6045
  Epoch 30/30 | Train Loss: 0.9960 Acc: 0.5753 | Val Loss: 0.9430 Acc: 0.592
5 | Best: 0.6045

Evaluating 09a...
  Accuracy: 0.5848 | F1: 0.5604
```

In [8]:

```python
# ── 09b: BN 7×7 ──────────────────────────────────────────────────────────
model_b, history_b = train('09b', CONFIGS['09b'])
print('\nEvaluating 09b...')
results_b = evaluate(model_b, CONFIGS['09b'])
```

```
[09b] BN 7×7 | No Pretrain
──────────────────────────────────────────────────────────────────────────────
  Epoch 01/30 | Train Loss: 1.7142 Acc: 0.3103 | Val Loss: 1.5199 Acc: 0.379
1 | Best: 0.3791
  Epoch 02/30 | Train Loss: 1.6248 Acc: 0.3700 | Val Loss: 1.4360 Acc: 0.453
7 | Best: 0.4537
  Epoch 03/30 | Train Loss: 1.3251 Acc: 0.4560 | Val Loss: 1.4914 Acc: 0.437
3 | Best: 0.4537
  Epoch 04/30 | Train Loss: 1.3243 Acc: 0.4545 | Val Loss: 1.4703 Acc: 0.450
7 | Best: 0.4537
  Epoch 05/30 | Train Loss: 1.2964 Acc: 0.4595 | Val Loss: 1.1955 Acc: 0.489
6 | Best: 0.4896
  Epoch 06/30 | Train Loss: 1.2500 Acc: 0.4664 | Val Loss: 1.2502 Acc: 0.471
6 | Best: 0.4896
  Epoch 07/30 | Train Loss: 1.2746 Acc: 0.4769 | Val Loss: 1.7581 Acc: 0.413
4 | Best: 0.4896
  Epoch 08/30 | Train Loss: 1.2365 Acc: 0.4843 | Val Loss: 1.7746 Acc: 0.256
7 | Best: 0.4896
  Epoch 09/30 | Train Loss: 1.2239 Acc: 0.4754 | Val Loss: 1.1759 Acc: 0.449
3 | Best: 0.4896
  Epoch 10/30 | Train Loss: 1.1862 Acc: 0.4734 | Val Loss: 1.1327 Acc: 0.516
4 | Best: 0.5164
  Epoch 11/30 | Train Loss: 1.2024 Acc: 0.4883 | Val Loss: 1.0927 Acc: 0.555
2 | Best: 0.5552
  Epoch 12/30 | Train Loss: 1.1492 Acc: 0.5062 | Val Loss: 1.0799 Acc: 0.553
7 | Best: 0.5552
  Epoch 13/30 | Train Loss: 1.1264 Acc: 0.5341 | Val Loss: 1.1015 Acc: 0.529
9 | Best: 0.5552
  Epoch 14/30 | Train Loss: 1.1411 Acc: 0.5127 | Val Loss: 1.0371 Acc: 0.564
2 | Best: 0.5642
  Epoch 15/30 | Train Loss: 1.1176 Acc: 0.5167 | Val Loss: 1.4237 Acc: 0.432
8 | Best: 0.5642
  Epoch 16/30 | Train Loss: 1.0819 Acc: 0.5435 | Val Loss: 1.0676 Acc: 0.540
3 | Best: 0.5642
  Epoch 17/30 | Train Loss: 1.0903 Acc: 0.5336 | Val Loss: 1.1404 Acc: 0.491
0 | Best: 0.5642
  Epoch 18/30 | Train Loss: 1.0824 Acc: 0.5405 | Val Loss: 0.9738 Acc: 0.600
0 | Best: 0.6000
  Epoch 19/30 | Train Loss: 1.0501 Acc: 0.5559 | Val Loss: 0.9345 Acc: 0.611
9 | Best: 0.6119
  Epoch 20/30 | Train Loss: 1.0255 Acc: 0.5719 | Val Loss: 0.9418 Acc: 0.610
4 | Best: 0.6119
  Epoch 21/30 | Train Loss: 1.0594 Acc: 0.5564 | Val Loss: 0.9161 Acc: 0.632
8 | Best: 0.6328
  Epoch 22/30 | Train Loss: 0.9954 Acc: 0.5808 | Val Loss: 0.9375 Acc: 0.607
5 | Best: 0.6328
  Epoch 23/30 | Train Loss: 1.0144 Acc: 0.5878 | Val Loss: 0.9752 Acc: 0.595
5 | Best: 0.6328
  Epoch 24/30 | Train Loss: 0.9940 Acc: 0.5967 | Val Loss: 0.8965 Acc: 0.641
8 | Best: 0.6418
  Epoch 25/30 | Train Loss: 0.9912 Acc: 0.5853 | Val Loss: 0.9355 Acc: 0.613
4 | Best: 0.6418
  Epoch 26/30 | Train Loss: 0.9865 Acc: 0.5922 | Val Loss: 0.8763 Acc: 0.643
3 | Best: 0.6433
  Epoch 27/30 | Train Loss: 0.9756 Acc: 0.5997 | Val Loss: 0.8528 Acc: 0.673
1 | Best: 0.6731
```

```
  Epoch 28/30 | Train Loss: 0.9579 Acc: 0.6072 | Val Loss: 0.8658 Acc: 0.653
7 | Best: 0.6731
  Epoch 29/30 | Train Loss: 0.9526 Acc: 0.5977 | Val Loss: 0.8608 Acc: 0.656
7 | Best: 0.6731
  Epoch 30/30 | Train Loss: 0.9715 Acc: 0.5997 | Val Loss: 0.8609 Acc: 0.655
2 | Best: 0.6731

Evaluating 09b...
  Accuracy: 0.6696 | F1: 0.6568
```

In [9]:

```python
# ── 09c: LN 3×3 ──────────────────────────────────────────────────────
model_c, history_c = train('09c', CONFIGS['09c'])
print('\nEvaluating 09c...')
results_c = evaluate(model_c, CONFIGS['09c'])
```

```
[09c] LN 3×3 | No Pretrain
──────────────────────────────────────────────────────────────────────────────
  Epoch 01/30 | Train Loss: 1.6345 Acc: 0.3222 | Val Loss: 1.4355 Acc: 0.371
6 | Best: 0.3716
  Epoch 02/30 | Train Loss: 1.9139 Acc: 0.2959 | Val Loss: 1.4484 Acc: 0.413
4 | Best: 0.4134
  Epoch 03/30 | Train Loss: 1.4519 Acc: 0.3590 | Val Loss: 1.3588 Acc: 0.391
0 | Best: 0.4134
  Epoch 04/30 | Train Loss: 1.3901 Acc: 0.3923 | Val Loss: 1.6716 Acc: 0.323
9 | Best: 0.4134
  Epoch 05/30 | Train Loss: 1.4317 Acc: 0.3690 | Val Loss: 1.2075 Acc: 0.522
4 | Best: 0.5224
  Epoch 06/30 | Train Loss: 1.2996 Acc: 0.4336 | Val Loss: 1.1919 Acc: 0.519
4 | Best: 0.5224
  Epoch 07/30 | Train Loss: 1.2884 Acc: 0.4311 | Val Loss: 1.2828 Acc: 0.382
1 | Best: 0.5224
  Epoch 08/30 | Train Loss: 1.2517 Acc: 0.4580 | Val Loss: 1.1377 Acc: 0.534
3 | Best: 0.5343
  Epoch 09/30 | Train Loss: 1.3073 Acc: 0.4237 | Val Loss: 1.2084 Acc: 0.456
7 | Best: 0.5343
  Epoch 10/30 | Train Loss: 1.2089 Acc: 0.4759 | Val Loss: 1.1836 Acc: 0.510
4 | Best: 0.5343
  Epoch 11/30 | Train Loss: 1.2520 Acc: 0.4436 | Val Loss: 1.1027 Acc: 0.531
3 | Best: 0.5343
  Epoch 12/30 | Train Loss: 1.1970 Acc: 0.4809 | Val Loss: 1.0658 Acc: 0.553
7 | Best: 0.5537
  Epoch 13/30 | Train Loss: 1.1866 Acc: 0.4888 | Val Loss: 1.0734 Acc: 0.567
2 | Best: 0.5672
  Epoch 14/30 | Train Loss: 1.1810 Acc: 0.4853 | Val Loss: 1.0846 Acc: 0.510
4 | Best: 0.5672
  Epoch 15/30 | Train Loss: 1.1269 Acc: 0.5147 | Val Loss: 1.0673 Acc: 0.541
8 | Best: 0.5672
  Epoch 16/30 | Train Loss: 1.1376 Acc: 0.5042 | Val Loss: 1.0341 Acc: 0.574
6 | Best: 0.5746
  Epoch 17/30 | Train Loss: 1.1185 Acc: 0.5077 | Val Loss: 1.0646 Acc: 0.529
9 | Best: 0.5746
  Epoch 18/30 | Train Loss: 1.0796 Acc: 0.5241 | Val Loss: 1.0350 Acc: 0.553
7 | Best: 0.5746
  Epoch 19/30 | Train Loss: 1.0548 Acc: 0.5271 | Val Loss: 1.0163 Acc: 0.552
2 | Best: 0.5746
  Epoch 20/30 | Train Loss: 1.0436 Acc: 0.5500 | Val Loss: 0.9780 Acc: 0.558
2 | Best: 0.5746
  Epoch 21/30 | Train Loss: 1.0593 Acc: 0.5306 | Val Loss: 0.9775 Acc: 0.561
2 | Best: 0.5746
  Epoch 22/30 | Train Loss: 1.0153 Acc: 0.5584 | Val Loss: 0.9811 Acc: 0.538
8 | Best: 0.5746
  Epoch 23/30 | Train Loss: 1.0233 Acc: 0.5415 | Val Loss: 0.9972 Acc: 0.544
8 | Best: 0.5746
  Epoch 24/30 | Train Loss: 1.0214 Acc: 0.5510 | Val Loss: 0.9511 Acc: 0.580
6 | Best: 0.5806
  Epoch 25/30 | Train Loss: 1.0110 Acc: 0.5505 | Val Loss: 0.9394 Acc: 0.594
0 | Best: 0.5940
  Epoch 26/30 | Train Loss: 0.9838 Acc: 0.5743 | Val Loss: 0.9412 Acc: 0.582
1 | Best: 0.5940
  Epoch 27/30 | Train Loss: 0.9911 Acc: 0.5733 | Val Loss: 0.9404 Acc: 0.579
1 | Best: 0.5940
```

```
  Epoch 28/30 | Train Loss: 1.0063 Acc: 0.5634 | Val Loss: 0.9374 Acc: 0.579
1 | Best: 0.5940
  Epoch 29/30 | Train Loss: 0.9824 Acc: 0.5704 | Val Loss: 0.9368 Acc: 0.568
7 | Best: 0.5940
  Epoch 30/30 | Train Loss: 0.9734 Acc: 0.5888 | Val Loss: 0.9361 Acc: 0.573
1 | Best: 0.5940

Evaluating 09c...
  Accuracy: 0.5625 | F1: 0.5354
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1
565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 i
n labels with no predicted samples. Use `zero_division` parameter to control
this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1
565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 i
n labels with no predicted samples. Use `zero_division` parameter to control
this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1
565: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 i
n labels with no predicted samples. Use `zero_division` parameter to control
this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", len(result))
```

In [10]:

```python
# ── 09d: BN 3×3 ──────────────────────────────────────────────────────────
model_d, history_d = train('09d', CONFIGS['09d'])
print('\nEvaluating 09d...')
results_d = evaluate(model_d, CONFIGS['09d'])
```

```
[09d] BN 3×3 | No Pretrain
──────────────────────────────────────────────────────────────────────────────
  Epoch 01/30 | Train Loss: 1.6729 Acc: 0.3357 | Val Loss: 1.6182 Acc: 0.432
8 | Best: 0.4328
  Epoch 02/30 | Train Loss: 1.5831 Acc: 0.3958 | Val Loss: 4.1919 Acc: 0.291
0 | Best: 0.4328
  Epoch 03/30 | Train Loss: 1.3400 Acc: 0.4550 | Val Loss: 1.1684 Acc: 0.537
3 | Best: 0.5373
  Epoch 04/30 | Train Loss: 1.3316 Acc: 0.4416 | Val Loss: 1.5678 Acc: 0.382
1 | Best: 0.5373
  Epoch 05/30 | Train Loss: 1.2562 Acc: 0.4893 | Val Loss: 1.3087 Acc: 0.438
8 | Best: 0.5373
  Epoch 06/30 | Train Loss: 1.2603 Acc: 0.4799 | Val Loss: 2.0203 Acc: 0.349
3 | Best: 0.5373
  Epoch 07/30 | Train Loss: 1.2120 Acc: 0.4938 | Val Loss: 1.2139 Acc: 0.552
2 | Best: 0.5522
  Epoch 08/30 | Train Loss: 1.2309 Acc: 0.4868 | Val Loss: 1.2104 Acc: 0.564
2 | Best: 0.5642
  Epoch 09/30 | Train Loss: 1.1771 Acc: 0.5047 | Val Loss: 1.0881 Acc: 0.589
6 | Best: 0.5896
  Epoch 10/30 | Train Loss: 1.1526 Acc: 0.5201 | Val Loss: 1.3578 Acc: 0.456
7 | Best: 0.5896
  Epoch 11/30 | Train Loss: 1.1274 Acc: 0.5137 | Val Loss: 1.0188 Acc: 0.594
0 | Best: 0.5940
  Epoch 12/30 | Train Loss: 1.1003 Acc: 0.5346 | Val Loss: 1.0065 Acc: 0.555
2 | Best: 0.5940
  Epoch 13/30 | Train Loss: 1.0755 Acc: 0.5475 | Val Loss: 1.0685 Acc: 0.532
8 | Best: 0.5940
  Epoch 14/30 | Train Loss: 1.0796 Acc: 0.5430 | Val Loss: 1.0745 Acc: 0.555
2 | Best: 0.5940
  Epoch 15/30 | Train Loss: 1.0769 Acc: 0.5286 | Val Loss: 1.1075 Acc: 0.531
3 | Best: 0.5940
  Epoch 16/30 | Train Loss: 1.0635 Acc: 0.5485 | Val Loss: 1.0469 Acc: 0.552
2 | Best: 0.5940
  Epoch 17/30 | Train Loss: 1.0359 Acc: 0.5684 | Val Loss: 1.0149 Acc: 0.579
1 | Best: 0.5940
  Epoch 18/30 | Train Loss: 1.0353 Acc: 0.5639 | Val Loss: 1.1035 Acc: 0.526
9 | Best: 0.5940
  Epoch 19/30 | Train Loss: 1.0410 Acc: 0.5649 | Val Loss: 1.1855 Acc: 0.471
6 | Best: 0.5940
  Epoch 20/30 | Train Loss: 1.0117 Acc: 0.5828 | Val Loss: 0.9129 Acc: 0.619
4 | Best: 0.6194
  Epoch 21/30 | Train Loss: 0.9833 Acc: 0.5922 | Val Loss: 0.9402 Acc: 0.628
4 | Best: 0.6284
  Epoch 22/30 | Train Loss: 0.9927 Acc: 0.5803 | Val Loss: 0.9482 Acc: 0.611
9 | Best: 0.6284
  Epoch 23/30 | Train Loss: 0.9782 Acc: 0.5738 | Val Loss: 0.8886 Acc: 0.626
9 | Best: 0.6284
  Epoch 24/30 | Train Loss: 0.9738 Acc: 0.6007 | Val Loss: 0.9423 Acc: 0.592
5 | Best: 0.6284
  Epoch 25/30 | Train Loss: 0.9676 Acc: 0.5967 | Val Loss: 0.8966 Acc: 0.635
8 | Best: 0.6358
  Epoch 26/30 | Train Loss: 0.9768 Acc: 0.5972 | Val Loss: 0.8746 Acc: 0.631
3 | Best: 0.6358
  Epoch 27/30 | Train Loss: 0.9543 Acc: 0.6091 | Val Loss: 0.8765 Acc: 0.647
8 | Best: 0.6478
```

```
  Epoch 28/30 | Train Loss: 0.9293 Acc: 0.6285 | Val Loss: 0.8612 Acc: 0.668
7 | Best: 0.6687
  Epoch 29/30 | Train Loss: 0.9297 Acc: 0.6151 | Val Loss: 0.8665 Acc: 0.655
2 | Best: 0.6687
  Epoch 30/30 | Train Loss: 0.9547 Acc: 0.6057 | Val Loss: 0.8624 Acc: 0.658
2 | Best: 0.6687

Evaluating 09d...
  Accuracy: 0.6310 | F1: 0.6236
```

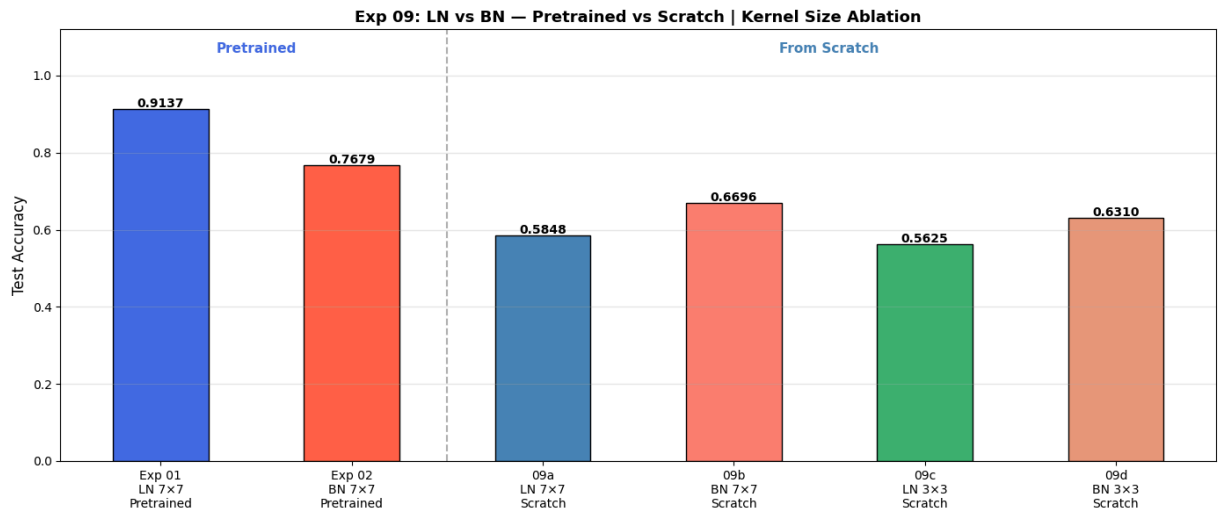
In [ ]:

```
# ── Summary table + gap analysis ──────────────────────────────────────
EXP01_ACC, EXP02_ACC = 0.9137, 0.7679   # pretrained LN vs BN gap reference

rows = [
    ('Exp 01 (LN 7×7, Pretrained)', EXP01_ACC, None),
    ('Exp 02 (BN 7×7, Pretrained)', EXP02_ACC, None),
    ('09a    (LN 7×7, Scratch)',     results_a['accuracy'], results_a['f1'])
    ('09b    (BN 7×7, Scratch)',     results_b['accuracy'], results_b['f1'])
    ('09c    (LN 3×3, Scratch)',     results_c['accuracy'], results_c['f1'])
    ('09d    (BN 3×3, Scratch)',     results_d['accuracy'], results_d['f1'])
]

print(f'{"Config":<35} {"Accuracy":>10} {"F1":>10}')
print('─' * 57)
for name, acc, f1 in rows:
    f1_str = f'{f1:.4f}' if f1 else '-'
    print(f'{name:<35} {acc:.4f}     {f1_str:>10}')

print('─' * 57)
print(f'\nGap analysis:')
print(f'  Pretrained  LN vs BN (7×7):  {EXP01_ACC - EXP02_ACC:+.4f}  (exp 01
print(f'  Scratch     LN vs BN (7×7):  {results_a["accuracy"] - results_b["a
print(f'  Scratch     LN vs BN (3×3):  {results_c["accuracy"] - results_d["a
print(f'  Scratch     LN 7×7 vs 3×3:  {results_a["accuracy"] - results_c["ac
print(f'  Scratch     BN 7×7 vs 3×3:  {results_b["accuracy"] - results_d["ac
```

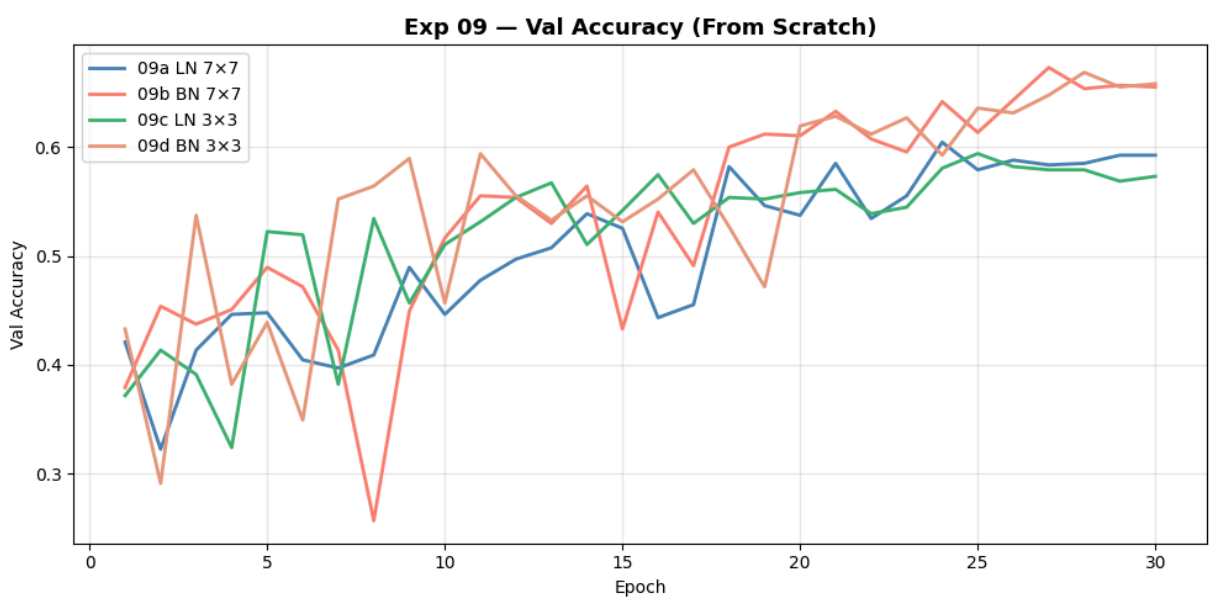
In [12]:

```
# ── Bar chart ──────────────────────────────────────────────────────────
labels = ['Exp 01\nLN 7×7\nPretrained', 'Exp 02\nBN 7×7\nPretrained',
          '09a\nLN 7×7\nScratch', '09b\nBN 7×7\nScratch',
          '09c\nLN 3×3\nScratch', '09d\nBN 3×3\nScratch']
accs   = [EXP01_ACC, EXP02_ACC,
          results_a['accuracy'], results_b['accuracy'],
          results_c['accuracy'], results_d['accuracy']]
colors = ['royalblue', 'tomato', 'steelblue', 'salmon', 'mediumseagreen', 'c

fig, ax = plt.subplots(figsize=(14, 6))
bars = ax.bar(labels, accs, color=colors, edgecolor='black', width=0.5)
for bar, acc in zip(bars, accs):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
            f'{acc:.4f}', ha='center', fontsize=10, fontweight='bold')
ax.axvline(x=1.5, color='gray', ls='--', alpha=0.6)
ax.text(0.5, 1.06, 'Pretrained', ha='center', fontsize=11, color='royalblue'
ax.text(3.5, 1.06, 'From Scratch', ha='center', fontsize=11, color='steelblu
ax.set_ylim(0, 1.12)
ax.set_ylabel('Test Accuracy', fontsize=12)
ax.set_title('Exp 09: LN vs BN — Pretrained vs Scratch | Kernel Size Ablatic
```

```
ax.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.savefig('comparison_bar_09.png', dpi=150)
plt.show()
```

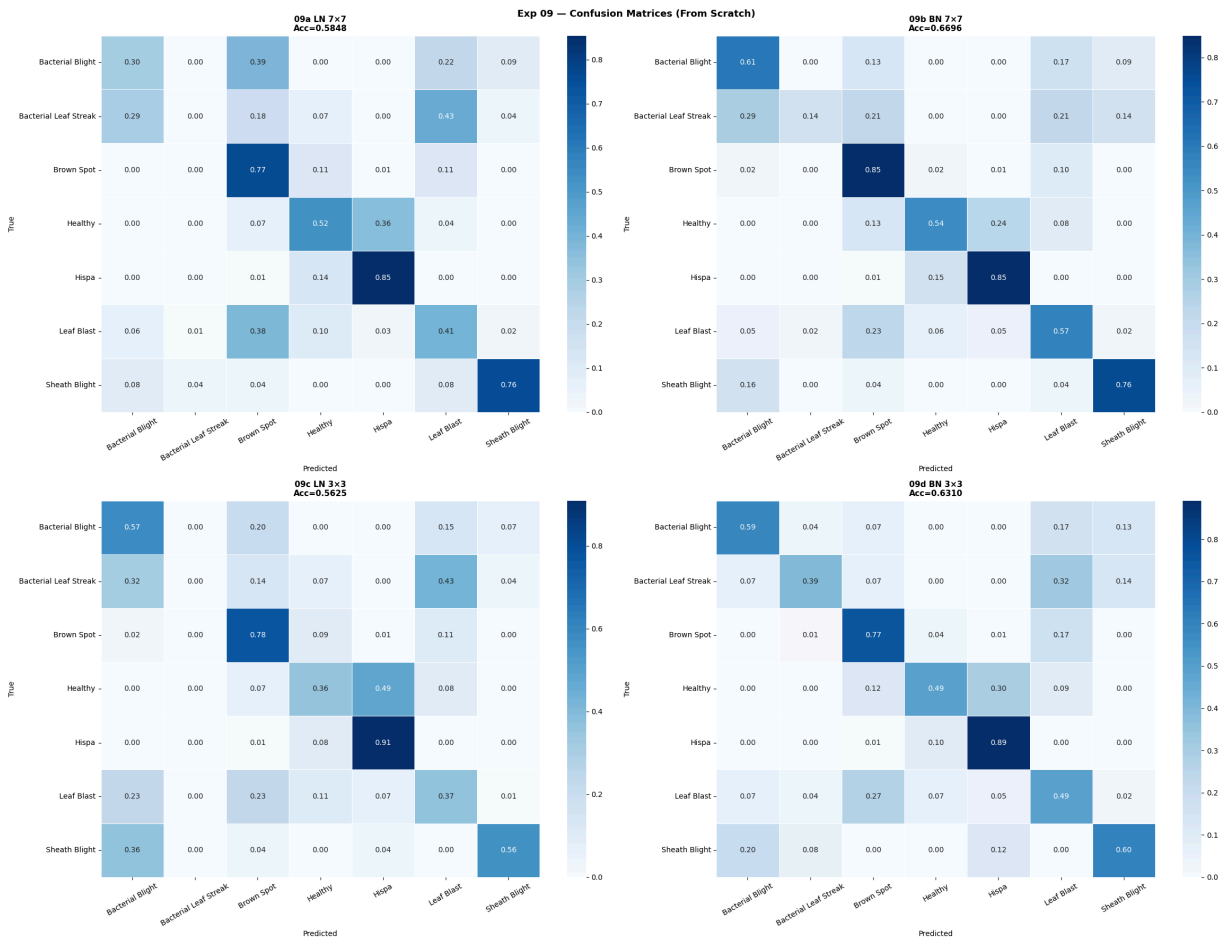


In [13]:
```
# — Overlaid val accuracy curves ————————————————————————
ep = range(1, NUM_EPOCHS + 1)
plt.figure(figsize=(10, 5))
plt.plot(ep, history_a['val_acc'], label='09a LN 7×7', color='steelblue',
plt.plot(ep, history_b['val_acc'], label='09b BN 7×7', color='salmon',
plt.plot(ep, history_c['val_acc'], label='09c LN 3×3', color='mediumseagreer
plt.plot(ep, history_d['val_acc'], label='09d BN 3×3', color='darksalmon',
plt.xlabel('Epoch'); plt.ylabel('Val Accuracy')
plt.title('Exp 09 — Val Accuracy (From Scratch)', fontsize=13, fontweight='b
plt.legend(); plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig('comparison_curves_09.png', dpi=150)
plt.show()
```



In [14]:
```
# — 4-way confusion matrices ————————————————————————————
fig, axes = plt.subplots(2, 2, figsize=(24, 18))
```

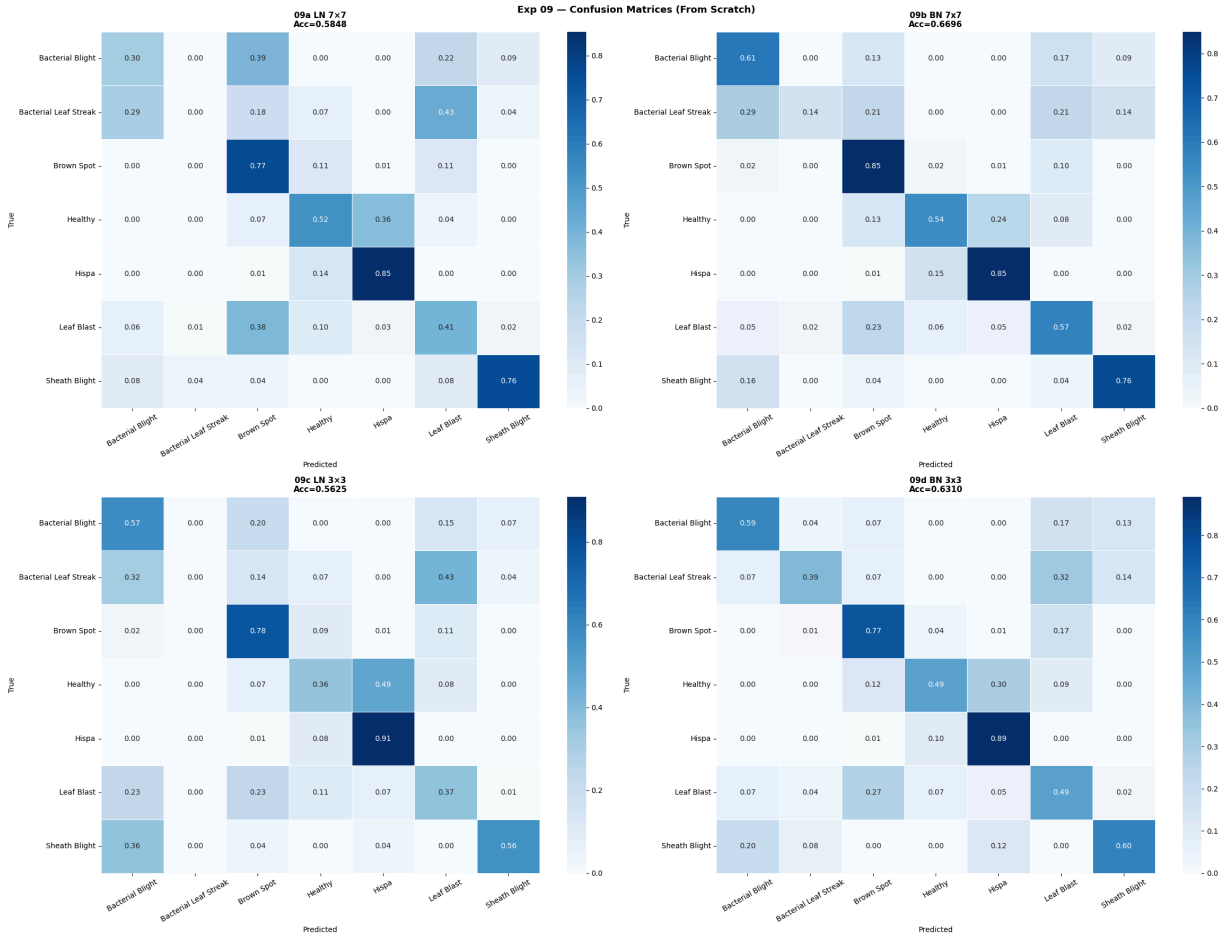
```
for ax, res, cfg in zip(axes.flatten(),
                        [results_a, results_b, results_c, results_d],
                        ['09a LN 7×7', '09b BN 7×7', '09c LN 3×3', '09d BN 3
    sns.heatmap(res['cm_norm'], annot=True, fmt='.2f',
                xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES,
                cmap='Blues', linewidths=0.5, ax=ax)
    ax.set_title(f'{cfg}\nAcc={res["accuracy"]:.4f}', fontsize=11, fontweigh
    ax.set_ylabel('True'); ax.set_xlabel('Predicted')
    ax.tick_params(axis='x', rotation=30)
fig.suptitle('Exp 09 — Confusion Matrices (From Scratch)', fontsize=13, font
plt.tight_layout()
plt.savefig('comparison_cm_09.png', dpi=150)
plt.show()
```



In [15]:

```
# ── 4-way confusion matrices ──────────────────────────────────────
fig, axes = plt.subplots(2, 2, figsize=(24, 18))
for ax, res, cfg in zip(axes.flatten(),
                        [results_a, results_b, results_c, results_d],
                        ['09a LN 7×7', '09b BN 7x7', '09c LN 3×3', '09d BN 3
    sns.heatmap(res['cm_norm'], annot=True, fmt='.2f',
                xticklabels=CLASS_NAMES, yticklabels=CLASS_NAMES,
                cmap='Blues', linewidths=0.5, ax=ax)
    ax.set_title(f'{cfg}\nAcc={res["accuracy"]:.4f}', fontsize=11, fontweigh
    ax.set_ylabel('True'); ax.set_xlabel('Predicted')
    ax.tick_params(axis='x', rotation=30)
fig.suptitle('Exp 09 — Confusion Matrices (From Scratch)', fontsize=13, font
plt.tight_layout()
```

```
plt.savefig('comparison_cm_09.png', dpi=150)
plt.show()
```



7. Save to Drive

In [16]:
```
for fname in ['comparison_bar_09.png', 'comparison_curves_09.png', 'comparis
    if os.path.exists(fname):
        shutil.copy(fname, f'{DRIVE_DIR}/{fname}')
        print(f'✅  {fname}')

# Save per-config classification reports
import pandas as pd
for key, res in [('09a', results_a), ('09b', results_b), ('09c', results_c),
    df = pd.DataFrame(res['report']).transpose()
    fname = f'classification_report_{key}.csv'
    df.to_csv(fname)
    shutil.copy(fname, f'{DRIVE_DIR}/{fname}')
    print(f'✅  {fname}')

print('\nDone.')
```

```
✅ comparison_bar_09.png
✅ comparison_curves_09.png
✅ comparison_cm_09.png
✅ classification_report_09a.csv
✅ classification_report_09b.csv
✅ classification_report_09c.csv
✅ classification_report_09d.csv

Done.
```